

JPA 3.0 & HIBERNATE 6 CHEAT SHEET

Enterprise Java Persistence

Kompaktreferenz: Mappings, Entity Graphs, Bulk Operations, Locking, Performance & Best Practices

Jakarta Persistence 3.0 | Hibernate ORM 6.6 | Derby / RDBMS

Entity Lifecycle & Zustandsmodell

NEW / TRANSIENT

Mit new erzeugt. Keine DB-ID, dem Persistence Context vollkommen unbekannt.

MANAGED

Im Persistence Context registriert. Dirty Checking persistiert Änderungen automatisch.

DETACHED

Besitzt DB-ID, aber nicht mehr im Persistence Context (nach close, clear oder Tx-Ende).

REMOVED

Mit em.remove() zum Löschen vorgemerkt. Wird beim nächsten flush/commit gelöscht.

KERNOPERATIONEN DES ENTITYMANAGERS

```
em.persist(vehicle);           // Transient -> Managed (Insert bei Commit/Flush)
Vehicle v = em.find(Vehicle.class, 1L); // DB -> Managed
Vehicle managed = em.merge(detachedV); // Detached -> liefert NEUE gemanagte Instanz!
em.remove(managed);           // Managed -> Removed (Delete bei Commit/Flush)
em.flush();                   // Schreibt anstehende SQL-Änderungen in die DB
em.clear();                   // Leert gesamten Cache (alle Entities werden Detached)
em.refresh(managed);          // Überschreibt Entity mit aktuellem DB-Stand
```

2.1 Primärschlüssel (Primary Key Generation)

Strategie	DB-Mechanismus	Vorteile & Verhalten	Nachteile / Fallstricke
IDENTITY	AUTO_INCREMENT / SERIAL	Einfach; DB generiert ID beim INSERT	Verhindert JDBC-Batch-Inserts (INSERT muss sofort feuern)
SEQUENCE (Empfohlen)	DB Sequence Objekt	Sehr schnell; Vorab-Allokation; Batch-Inserts voll unterstützt	DB muss Sequenzen unterstützen (Oracle, PG, Derby)
TABLE	Eigene Generator-Tabelle	Portabel über alle DBs	Langsam; sperrt Hilfstabelle bei jedem Key-Fetch
AUTO	Provider-Wahl	Wählt SEQUENCE oder IDENTITY	Nicht deterministisch über verschiedene Provider

BEST PRACTICE: SEQUENCE MIT ALLOCATION SIZE

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "veh_seq_gen")
@SequenceGenerator(name = "veh_seq_gen", sequenceName = "SEQ_VEHICLE", allocationSize = 50)
private Long id;
```

2.2 Zusammengesetzte Primärschlüssel (@EmbeddedId)

EMBEDDED ID IMPLEMENTIERUNG

```
@Embeddable
public class BookingId implements Serializable {
    private String contractNumber;
    private String branchCode;

    // Default-Konstruktor, Konstruktor mit Feldern
    // equals() und hashCode() basierend auf beiden Feldern!
}

@Entity
public class Booking {
    @EmbeddedId
    private BookingId id;

    private String customerName;
    private double totalAmount;
}
```

Best Practice Regeln für Composite Keys

- 1. @Embeddable & Serializable: Die Schlüsselklasse muss zwingend Serializable implementieren und einen parameterlosen Konstruktor besitzen.
- 2. Konsistente equals() und hashCode(): Essentiell für den First-Level-Cache des EntityManagers und Set-Operationen.
- 3. @EmbeddedId vs @IdClass: @EmbeddedId kapselt die Identität in einem sauberen Wertobjekt. @IdClass wiederholt Attribute flach in der Entity und ist fehleranfällig.
- 4. Abfrage per ID: em.find(Booking.class, new BookingId("RENT-001", "BER-01")) ist typsicher und kompakt.

2.3 Wertobjekte: @Embeddable & @Embedded

EMBEDDABLE & ATTRIBUTE OVERRIDES

```
@Embeddable
public class Address {
    private String street;
    private String zipCode;
    private String city;
}

@Entity
public class User {
    @Id @GeneratedValue private Long id;

    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "street", column = @Column(name =
"BILLING_STREET")),
        @AttributeOverride(name = "zipCode", column = @Column(name =
"BILLING_ZIP"))
    })
    private Address billingAddress;
}
```

Eigenschaften von Embeddables

- Keine eigene Identität: Besitzen keinen eigenen @Id-Schlüssel.
- Gemeinsame Tabelle: Felder werden direkt in die Tabelle der besitzenden Entity gemappt.
- AttributeOverrides: Ermöglicht das mehrfache Einbetten desselben Wertobjekts (z.B. billingAddress und shippingAddress) mit unterschiedlichen Spaltennamen.
- Nested Embeddables: Embeddables können wiederum weitere Embeddables enthalten.

2.4 Benutzerdefinierte Typen mit @Converter

ATTRIBUTE CONVERTER DEFINITION

```
public enum FuelType {
    PETROL("P"), DIESEL("D"), ELECTRIC("E"), HYBRID("H");
    private final String code;
    FuelType(String c) { this.code = c; }
    public String getCode() { return code; }
    public static FuelType fromCode(String c) { ... }
}

@Converter(autoApply = true)
public class FuelTypeConverter implements AttributeConverter<FuelType,
String> {
    public String convertToDatabaseColumn(FuelType type) {
        return type != null ? type.getCode() : null;
    }
    public FuelType convertToEntityAttribute(String code) {
        return FuelType.fromCode(code);
    }
}
```

Warum @Converter statt @Enumerated?

- 1. Schutz vor Datenkorruption: ORDINAL speichert 0, 1, 2. Wird das Enum erweitert oder umsortiert, ändern sich alle Bedeutungen in der DB!
- 2. Maximale Speichereffizienz: Speichert kompakte Einzelzeichen ('E', 'D', 'P') statt langer String-Texte.
- 3. autoApply = true: Bindet den Konverter automatisch an alle Entities, die diesen Typ verwenden (kein manuelles @Convert nötig).
- 4. Beliebige Java-Typen: Auch für verschlüsselte Strings, JSON-Payloads, Geldbeträge oder Custom Value Objects geeignet.

3.1 Basis-Beziehungen (1:1 & N:1)

1:1 UND N:1 RELATIONSMAPPINGS

```
// N:1 Beziehung (Owning Side):
@Entity
public class Vehicle {
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "SHOP_ID")
    private Shop shop;
}

// 1:1 Beziehung mit Fremdschlüssel:
@Entity
public class User {
    @OneToOne(fetch = FetchType.LAZY, cascade = CascadeType.ALL)
    @JoinColumn(name = "DRIVER_LICENSE_ID", unique = true)
    private DriverLicense license;
}
```

Best Practices für To-One Relationen

- Immer FetchType.LAZY: JPA Default ist EAGER auf @ManyToOne und @OneToOne. Unbedingt explizit auf LAZY überschreiben!
- Owning Side: Die Seite mit @JoinColumn hält die physische Fremdschlüsselspalte in der Tabelle.
- Unique Constraint bei 1:1: Verhindert Mehrfachzuweisungen desselben Kindes auf Datenbankebene.

3.2 Collection-Beziehungen (1:N)

BIDIREKTIONALE 1:N BEZIEHUNG

```
@Entity
public class Shop {
    @OneToMany(mappedBy = "shop",
                cascade = CascadeType.ALL,
                orphanRemoval = true)
    private List<Vehicle> carpool = new ArrayList<>();

    // Defensive Synchronisation:
    public void addVehicle(Vehicle v) {
        carpool.add(v);
        v.setShop(this);
    }
    public void removeVehicle(Vehicle v) {
        carpool.remove(v);
        v.setShop(null);
    }
}
```

Regeln für 1:N Beziehungen

- mappedBy ist Pflicht: Verweist auf den Java-Feldnamen der Gegenseite (@ManyToOne private Shop shop;). Ohne mappedBy erzeugt JPA eine unnötige Join-Tabelle!
- Initialisierung: Collections immer direkt bei der Deklaration mit new ArrayList<>() initialisieren.
- Defensive Methoden: addVehicle() und removeVehicle() halten beide Seiten im Speicher synchron.

3.3 Many-to-Many (@ManyToMany)

M:N MAPPING MIT JOIN-TABELLE

```
@Entity
public class Vehicle {
    @ManyToMany
    @JoinTable(
        name = "TBL_VEHICLE_FEATURE",
        joinColumns = @JoinColumn(name = "VEHICLE_ID"),
        inverseJoinColumns = @JoinColumn(name = "FEATURE_ID")
    )
    private Set<Feature> features = new HashSet<>();
}

@Entity
public class Feature {
    @ManyToMany(mappedBy = "features")
    private Set<Vehicle> vehicles = new HashSet<>();
}
```

Best Practices für M:N Beziehungen

- Set statt List: Verwende Set<T>, um Duplikate und ineffizientes Löschen in Join-Tabellen zu verhindern.
- Kein CascadeType.REMOVE: Löschen eines Fahrzeugs darf niemals das globale Ausstattungsmerkmal 'Klimaanlage' löschen!
- Verknüpfungsattribute: Falls die Beziehung eigene Attribute hat (z.B. assignedAt), als zwei 1:N Beziehungen mit eigener Verknüpfungs-Entity modellieren.

3.4 Kaskadierung & Orphan Removal (Parent-Child)

AGGREGAT-LIFECYCLE MIT ORPHAN REMOVAL

```
@Entity
public class RentalContract {
    @Id @GeneratedValue private Long id;

    @OneToMany(mappedBy = "contract",
                cascade = CascadeType.ALL,
                orphanRemoval = true)
    private List<DamageRecord> damages = new ArrayList<>();

    public void addDamage(DamageRecord d) {
        damages.add(d);
        d.setContract(this);
    }

    public void removeDamage(DamageRecord d) {
        damages.remove(d);
        d.setContract(null); // Triggert SQL DELETE!
    }
}
```

Unterschied: Cascade REMOVE vs Orphan Removal

- CascadeType.REMOVE: Löscht Kind-Datensätze nur dann, wenn das gesamte Elternobjekt (RentalContract) per em.remove() gelöscht wird.
- orphanRemoval = true: Löscht das Kindobjekt zusätzlich aus der DB, wenn es aus der Java-Collection entfernt wird (contract.removeDamage(d)).
- Echtes Aggregat: Perfekt für untergeordnete Details (Rechnungsposten, Schäden, Buchungszeilen).
- Verbot: Niemals auf unabhängigen Stammdaten (Fahrer, Standorte, Kategorien) anwenden!

4. Vererbungsstrategien (Inheritance)

Strategie	DB-Tabellen	Performance	Datenintegrität	Einsatzempfehlung
SINGLE_TABLE (Standard)	1 gemeinsame Tabelle	Sehr schnell (keine Joins)	Schwach (Subklassen-Spalten müssen NULL erlauben)	Flache Hierarchien mit wenigen Subklassen-Feldern
JOINED	1 Basis + 1 je Subklasse	Mittel (Joins über PK/FK nötig)	Exzellent (echte NOT NULL & FK-Constraints)	Tiefe Hierarchien mit vielen spezifischen Attributen
TABLE_PER_CLASS	1 je konkrete Klasse	Langsam bei Polymorphie (UNION nötig)	Gut je Einzeltabelle	Selten polymorphe Abfragen über Basisklasse
@MappedSuperclass	Keine Basistabelle	Sehr schnell (Attribute in Kindtabellen)	Gut je Entity	Gemeinsame technische Basisfelder (id, createdAt)

BEISPIEL: SINGLE_TABLE MIT DISKRIMINATOR

```

@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "DISC_TYPE", discriminatorType = DiscriminatorType.STRING)
public abstract class VehicleType { ... }

```

5.1 JPQL Abfragen & DTO Projektionen

JPQL ABFRAGEN UND PROJEKTIONEN

```
// 1. Einfache Abfrage mit benanntem Parameter:
TypedQuery<Vehicle> q1 = em.createQuery(
    "SELECT v FROM Vehicle v WHERE v.brand = :brand ORDER BY
    v.dailyRate ASC",
    Vehicle.class);
q1.setParameter("brand", "BMW");
List<Vehicle> list = q1.getResultList();

// 2. DTO Constructor Expression (Direkte Projektion):
TypedQuery<VehicleDto> q2 = em.createQuery(
    "SELECT new net.rentacar.dto.VehicleDto(v.brand, v.modell,
    v.dailyRate) " +
    "FROM Vehicle v WHERE v.active = true", VehicleDto.class);
List<VehicleDto> dtos = q2.getResultList();
```

JPQL Regeln & Best Practices

- Entity-Namen statt Tabellen: JPQL operiert auf Java-Klassen und Attributen, nicht auf SQL-Tabellen und Spalten.
- Benannte Parameter (:name): Verhindert SQL-Injections und ermöglicht Statement-Caching.
- DTO Constructor Expressions: Vermeidet das Laden kompletter Managed Entities in den Persistence Context für reine Lese-Operationen.
- Paging: setFirstResult(offset) und setMaxResults(limit) nutzen.

5.2 JPQL Joins & N+1 Vermeidung (JOIN FETCH)

JOIN TYPEN IN JPQL

```
// Inner Join (Filtert Shops ohne Fahrzeuge):  
SELECT s FROM Shop s JOIN s.carpool v WHERE v.active = true  
  
// Left Join (Liefert alle Shops):  
SELECT s FROM Shop s LEFT JOIN s.carpool v  
  
// JOIN FETCH (Lädt Assoziation in 1 SQL-Query eagerly!):  
SELECT DISTINCT s FROM Shop s  
LEFT JOIN FETCH s.carpool  
WHERE s.city = :city
```

Das N+1 Select Problem lösen

- N+1 Ursache: 1 Query lädt N Parent-Entities. Beim Zugriff auf die Lazy-Collection feuert JPA N weitere Queries ab.
- Lösung JOIN FETCH: Lädt die Parents und ihre Children in einem einzigen SQL-Join.
- DISTINCT Schlüsselwort: Verhindert Duplikate des Parents im Java-Ergebnis, wenn dieser mehrere Kind-Datensätze besitzt.
- Achtung: Niemals mehrere JOIN FETCH auf Collections in einer Query kombinieren (MultipleBagFetchException / Kartesisches Produkt)!

5.3 JPA Entity Graphs (fetchgraph & loadgraph)

ENTITY GRAPH API

```
// 1. Programmatischer EntityGraph
EntityGraph<Shop> graph = em.createEntityGraph(Shop.class);
graph.addAttributeNodes("carpool");
// Optional: Verschachtelter Subgraph
Subgraph<Vehicle> vehGraph = graph.addSubgraph("carpool");
vehGraph.addAttributeNodes("type");

// 2. Abfrage mit Query Hint
List<Shop> shops = em.createQuery("SELECT s FROM Shop s", Shop.class)
    .setHint("jakarta.persistence.fetchgraph", graph)
    .getResultList();
```

Vorteile & Hints im Überblick

- Dynamischer Ladeplan: Ändert nicht den JPQL-String; derselbe Basis-Query kann für Listen und Details verwendet werden.
- fetchgraph vs loadgraph:
 - fetchgraph: Alle gelisteten Attribute = EAGER, alle ungelisteten = LAZY (überschreibt statisches EAGER).
 - loadgraph: Gelistete Attribute = EAGER, ungelistete behalten ihr Mapping.
- Deklarativ per `@NamedEntityGraph` an der Entity oder dynamisch per `createEntityGraph()` im Code erstellbar.

5.4 Massenoperationen (Bulk Updates & Deletes)

BULK UPDATE WORKFLOW

```
// 1. Ausstehende Änderungen schreiben
em.flush();

// 2. Massenoperation direkt auf Datenbank ausführen
int updated = em.createQuery(
    "UPDATE BulkVehicle v SET v.dailyRate = v.dailyRate * 1.1 " +
    "WHERE v.active = true")
    .executeUpdate();

// 3. Cache leeren, um veralteten Zustand zu verhindern!
em.clear();
```

Die Stale-Data-Falle bei executeUpdate()

- Direktes SQL: executeUpdate() umgeht den Persistence Context (1st-Level Cache) vollständig.
- Inkonsistenz im Speicher: Bereits geladene Entities im Cache werden nicht aktualisiert oder gelöscht!
- Die 3-Schritt-Regel:
 - 1. flush(): Dirty Changes vor der Massenoperation in die DB schreiben.
 - 2. executeUpdate(): SQL UPDATE/DELETE auf DB absetzen.
 - 3. clear(): Cache leeren, damit Entities neu aus DB geladen werden.

6. Criteria API Abfragen

DYNAMISCHE CRITERIA QUERY

```
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Vehicle> cq = cb.createQuery(Vehicle.class);
Root<Vehicle> root = cq.from(Vehicle.class);

// Prädikate dynamisch zusammenbauen:
List<Predicate> predicates = new ArrayList<>();
if (brandFilter != null) {
    predicates.add(cb.equal(root.get("brand"), brandFilter));
}
predicates.add(cb.greaterThan(root.get("dailyRate"), 50.0));

cq.where(predicates.toArray(new Predicate[0]))
    .orderBy(cb.asc(root.get("dailyRate"))));

List<Vehicle> results = em.createQuery(cq).getResultList();
```

Einsatzbereiche der Criteria API

- Dynamische Suchformulare: Beliebige Kombinationen von Filterkriterien ohne String-Zusammenbau von JPQL
- Typsicherheit: In Kombination mit dem JPA Metamodel (Annotation Processor) vollständig refactoring-sicher (z.B. `root.get(Vehicle_.brand)`).
- Komplexität: Für statische Abfragen ist JPQL deutlich lesbarer und vorzuziehen.

7. Entity Lifecycle Callbacks & Listener

CALLBACK METHODEN & LISTENER

```
@EntityListeners(AuditListener.class)
@Entity
public class Vehicle {
    private Instant createdAt;
    private Instant updatedAt;

    @PrePersist
    public void onPrePersist() {
        this.createdAt = Instant.now();
    }

    @PreUpdate
    public void onPreUpdate() {
        this.updatedAt = Instant.now();
    }
}
```

Verfügbare Lifecycle Events

- @PrePersist / @PostPersist: Vor / nach dem INSERT in die DB.
- @PreUpdate / @PostUpdate: Vor / nach dem UPDATE in der DB.
- @PreRemove / @PostRemove: Vor / nach dem DELETE in der DB.
- @PostLoad: Nach dem Laden und Instanzieren der Entity aus der DB.
- EntityListener: Auslagerung in wiederverwendbare Klassen (z.B. für automatisches Auditing).

8.1 Concurrency: Optimistisches Locking (@Version)

OPTIMISTIC LOCKING MIT @VERSION

```
@Entity
public class Vehicle {
    @Id @GeneratedValue private Long id;

    @Version
    private Long version; // Automatische Versionierung

    private String brand;
    private boolean reserved;
}

// Hibernate generiert bei jedem UPDATE:
// UPDATE VEHICLE SET RESERVED = 1, VERSION = 2
// WHERE ID = 1 AND VERSION = 1
```

OptimisticLockException & Verhalten

- Verhindert Lost Updates: Erkennt, wenn ein Datensatz zwischen Lesen und Schreiben von einer anderen Transaktion geändert wurde.
- Keine DB-Sperren: Maximale Lese- und Schreib-Parallelität ohne Datenbank-Locks.
- Ausnahmebehandlung: Wirft OptimisticLockException beim commit(). Transaktion muss abgebrochen und ggf. wiederholt werden.
- Unterstützte Typen: Long, Integer, Short oder java.time.Instant.

8.2 Concurrency: Pessimistisches Locking (SELECT FOR UPDATE)

PESSIMISTIC LOCKING API

```
// 1. Beim Laden sperren (SELECT ... FOR UPDATE)
PessimisticVehicle v = em.find(
    PessimisticVehicle.class, 1L,
    LockModeType.PESSIMISTIC_WRITE);

// 2. Nachträglich sperren
em.lock(v, LockModeType.PESSIMISTIC_WRITE);

// 3. In JPQL Query mit Timeout
List<PessimisticVehicle> list = em.createQuery(
    "SELECT v FROM PessimisticVehicle v WHERE v.reserved = false",
    PessimisticVehicle.class)
    .setLockMode(LockModeType.PESSIMISTIC_WRITE)
    .setHint("jakarta.persistence.lock.timeout", 3000)
    .getResultList();
```

Wann Pessimistisches Locking?

- Hohe Schreibkollision: Bei knappen Ressourcen (Ticketverkauf, Fahrzeugbuchung, Kontostände), wo parallele Versuche fast sicher kollidieren.
- PESSIMISTIC_WRITE: Exklusiver Lock auf DB-Zeilenebene; blockiert parallele Schreib- und Lese-Locks anderer Transaktionen.
- PESSIMISTIC_READ: Shared Lock; verhindert parallele Updates.
- Lock-Timeout: Mit `jakarta.persistence.lock.timeout` konfigurieren, um ewiges Hängen bei Deadlocks zu verhindern.

9.1 Performance: Batching, Caching & Lazy Loading

BATCH KONFIGURATION & LOOP

```
// Batch-Inserts in persistence.xml aktivieren:
<property name="hibernate.jdbc.batch_size" value="50"/>
<property name="hibernate.order_inserts" value="true"/>
<property name="hibernate.order_updates" value="true"/>

// Java Batch Loop:
for (int i = 0; i < list.size(); i++) {
    em.persist(list.get(i));
    if (i > 0 && i % 50 == 0) {
        em.flush();
        em.clear();
    }
}
```

Die wichtigsten Performance-Hebel

- JDBC Batching: Kombiniert mehrere INSERTs/UPDATEs in einen einzigen Netzwerk-Roundtrip zur Datenbank.
- Flush & Clear Loop: Verhindert Speicherüberlauf (OutOfMemoryError) bei Massendatenverarbeitung.
- Second-Level-Cache (2LC): Cacht Lese-Entities über Transaktionsgrenzen hinweg (z.B. mit Caffeine oder Ehcache).
- Query-Statistiken: hibernate.generate_statistics=true liefert Metriken über Statement-Zahlen und Cache-Hits.

9.2 Entity Identity: Equals & hashCode

BUSINESS KEY IMPLEMENTIERUNG

```
@Entity
public class Vehicle {
    @Id @GeneratedValue private Long id;
    private String fin; // Eindeutige Fahrgestellnummer (Business Key)

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Vehicle that = (Vehicle) o;
        return Objects.equals(fin, that.fin); // Business Key
    }

    @Override
    public int hashCode() {
        return Objects.hash(fin);
    }
}
```

Regeln für equals() und hashCode()

- 1. Niemals nur @Id nutzen: Vor dem Persistieren ist id == null. Ändert sich der Hashcode nach dem INSERT, geht die Entity im HashSet verloren!
- 2. Bevorzuge natürliche Business Keys: Fachlich eindeutige und unveränderliche Attribute (z.B. FIN, IBAN, UUID).
- 3. Konstanter Hashcode als Fallback: Falls kein natürlicher Schlüssel existiert, getClass().hashCode() als fixer Hashcode und ID-Vergleich nur in equals().
- 4. Getter verwenden: In equals() immer getId() statt Direktzugriff nutzen, um Proxies korrekt zu unterstützen.

10. persistence.xml Konfigurationsvorlage

PERSISTENCE.XML TEMPLATE

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence" version="3.0">
  <persistence-unit name="rentacarPU" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

    <class>net.rentacar.model.Vehicle</class>
    <class>net.rentacar.model.Shop</class>

    <properties>
      <!-- JDBC Verbindungseinstellungen -->
      <property name="jakarta.persistence.jdbc.driver"
value="org.apache.derby.jdbc.ClientDriver"/>
      <property name="jakarta.persistence.jdbc.url"
value="jdbc:derby://localhost:1527/rentacar;create=true"/>
      <property name="jakarta.persistence.jdbc.user" value="APP"/>
      <property name="jakarta.persistence.jdbc.password" value="APP"/>

      <!-- Schema-Generierung: none | create | drop-and-create | drop -->
      <property name="jakarta.persistence.schema-
generation.database.action" value="drop-and-create"/>

      <!-- SQL-Diagnose & Formatierung -->
      <property name="hibernate.show_sql" value="true"/>
      <property name="hibernate.format_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

Wichtige Konfigurationsschlüssel

- Standard-Properties: Verwende moderne jakarta.persistence.* Properties anstelle alter javax.persistence.* Keys.
- Schema-Action: In Produktion immer auf none setzen; in Integrationstests drop-and-create oder create.
- hibernate.show_sql & format_sql: Unverzichtbar zur Analyse von N+1 Problemen und generierten Joins.
- Entity-Listing: Explizites Listen aller <class> Einträge verhindert Classpath-Scanning-Verzögerungen.

JPA Entwickler-Checkliste & Best Practices

Modellierung & Mappings

- ✓ Alle To-One (@ManyToOne, @OneToOne) explizit auf LAZY umgestellt?
- ✓ Defensive Helper-Methoden (add/remove) in Parent-Entities implementiert?
- ✓ orphanRemoval=true für echte Eltern-Kind-Aggregate eingerichtet?
- ✓ AttributeConverter (@Converter) für saubere Enums & Custom Types genutzt?
- ✓ @EmbeddedId als Standard für zusammengesetzte Primärschlüssel verwendet?
- ✓ equals() und hashCode() basieren auf Business Keys (nicht allein auf @Id)?

Abfragen & Concurrency

- ✓ JOIN FETCH oder Entity Graphs genutzt, um N+1 Probleme zu verhindern?
- ✓ DTO Constructor Expressions (SELECT new ...) für reine Lese-Views verwendet?
- ✓ Nach Massenoperationen (executeUpdate) immer em.clear() aufgerufen?
- ✓ Optimistic Locking (@Version) bei parallelen Schreibzugriffen aktiv?
- ✓ Pessimistic Locking (SELECT FOR UPDATE) für Hotspot-Ressourcen konfiguriert?
- ✓ Batching (flush/clear) bei großen Datenmengen implementiert?